LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 9
CONVOLUTIONAL NEURAL NETWORK



Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
Kelas	: PL5A1
Dosen Pengampu	: Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi **2**
Tujuan Praktikum **3**
Dasar Teori **4**
2.1 Representasi Citra Digital dan Keterbatasan MLP 4
2.2 Convolutional Neural Network (CNN) 4
2.2.1 Convolution Layer 4
2.2.2 Fungsi Aktivasi ReLU 5
2.2.3 Pooling Layer 5
2.2.4 Flattening dan Fully Connected Layer 5
2.3 Regularisasi Dropout 5
2.4 Fungsi Loss, Optimizer, dan Metrik Evaluasi 6
Alat dan Bahan **7**
3.1 Perangkat Keras 7
3.2 Perangkat Lunak 7
3.3 Dataset 7
Hasil dan Pembahasan **8**
4.1 Percobaan Modul: Klasifikasi Citra Dog vs Cat 8
4.1.1 Unduh Dataset 8
4.1.2 Impor Pustaka 8
4.1.3 Pembacaan dan Pembagian Dataset 9
4.1.4 Uji Coba Pembacaan Data 10
4.1.5 Cek Bentuk Data dan IMG_SHAPE 11
4.1.6 Membangun Model CNN Sederhana 11
4.1.7 Melatih Model 12
4.1.8 Evaluasi Performa Model 14
4.2 Tugas dan Analisis: Klasifikasi Multi-Kelas CIFAR-10 15
4.2.1 Memuat Dataset 15
4.2.2 Pembagian Data 16
4.2.3 Model Pembanding Tanpa Dropout 16
4.2.4 Model CNN dengan Regularisasi Dropout 18
4.2.5 Grafik Loss dan Akurasi 20
4.2.6 Evaluasi pada Data Uji 21
4.2.7 Confusion Matrix 22
4.2.8 Contoh Hasil Prediksi 23
Kesimpulan **24**

Tujuan Praktikum

Berdasarkan Modul Pertemuan 9 Praktikum Penambangan Data, tujuan praktikum ini adalah:

1. Memahami keterbatasan *Multi-Layer Perceptron* (MLP) saat mengolah citra digital dan keunggulan arsitektur *Convolutional Neural Network* (CNN).
2. Menjelaskan komponen dasar CNN, yaitu *convolution layer*, fungsi aktivasi ReLU, *pooling layer*, *flattening*, dan *fully connected layer* (*dense*).
3. Membangun model CNN sederhana menggunakan *framework* Keras dan TensorFlow.
4. Menerapkan regularisasi *dropout* untuk mencegah *overfitting*.

Dasar Teori

2.1 Representasi Citra Digital dan Keterbatasan MLP

Komputer membaca citra sebagai matriks piksel. Citra berwarna berukuran 224×224 disimpan sebagai *array* berdimensi $224 \times 224 \times 3$, dengan tiga kanal warna merah, hijau, dan biru (RGB). Setiap elemen bernilai bulat 0 sampai 255 yang menyatakan intensitas cahaya pada kanal tersebut [2]. Klasifikasi citra sendiri termasuk tugas klasifikasi dalam penambangan data, yaitu memetakan objek ke salah satu label kelas berdasarkan fitur yang dimilikinya [3].

Kalau citra seperti ini langsung dimasukkan ke MLP, ada dua masalah yang muncul. Masalah pertama adalah jumlah parameter yang membengkak (*parameter explosion*). Satu citra $224 \times 224 \times 3$ menghasilkan 150.528 nilai masukan, sehingga satu *hidden layer* berisi 1.000 neuron saja sudah membutuhkan sekitar 150 juta bobot. Masalah kedua adalah hilangnya informasi spasial. MLP mewajibkan citra diratakan menjadi vektor satu dimensi, sehingga hubungan ketetanggaan antarpiksel hilang. Akibatnya MLP mudah gagal ketika objek bergeser, berubah skala, atau berotasi sedikit [9].

2.2 Convolutional Neural Network (CNN)

CNN adalah jenis *feed-forward neural network* yang dirancang untuk data berbentuk *grid*, terutama citra. CNN belajar mengekstrak pola visual secara bertingkat langsung dari piksel mentah, mulai dari tepi dan tekstur pada lapisan awal sampai bentuk objek pada lapisan yang lebih dalam [10]. Ada dua gagasan yang membuat CNN hemat parameter: *local connectivity* (setiap neuron hanya melihat sebagian kecil citra) dan *weight sharing* (filter yang sama dipakai di seluruh posisi citra) [9].

2.2.1 Convolution Layer

Lapisan konvolusi memakai sejumlah filter atau *kernel* berukuran kecil, misalnya 3×3, yang digeser (*stride*) di atas citra masukan. Pada setiap posisi, filter menghitung perkalian titik (*dot product*) dengan potongan citra di bawahnya. Untuk citra masukan I dan kernel K

berukuran $k \times k$, nilai *feature map* pada posisi (i, j) adalah:

$$S(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i+m, j+n)\, K(m,n) + b \tag{1}$$

Setiap filter menghasilkan satu *feature map*. Jumlah parameter satu lapisan `Conv2D` dihitung dengan $(k \cdot k \cdot C_{in} + 1) \cdot C_{out}$, dengan C_{in} jumlah kanal masukan dan C_{out} jumlah filter. Opsi `padding='same'` menambahkan nilai nol di tepi citra agar ukuran keluaran sama dengan masukan [4].

2.2.2 Fungsi Aktivasi ReLU

Setelah konvolusi, nilai *feature map* dilewatkan ke fungsi aktivasi *Rectified Linear Unit* (ReLU):

$$\Phi(x) = \max(0, x) \tag{2}$$

ReLU mengubah semua nilai negatif menjadi nol dan meneruskan nilai positif apa adanya. Fungsi ini membuat jaringan mampu memodelkan hubungan non-linier dan komputasinya murah [8].

2.2.3 Pooling Layer

Pooling mengecilkan ukuran spasial (lebar dan tinggi) *feature map* sehingga beban komputasi berkurang dan model lebih tahan terhadap pergeseran kecil (*translation invariance*). Jenis yang paling sering dipakai adalah *max pooling*, yaitu mengambil nilai terbesar dalam jendela tertentu. *Max pooling* 2×2 memperkecil *feature map* 112×112 menjadi 56×56 tanpa menambah parameter [2].

2.2.4 Flattening dan Fully Connected Layer

Flattening mengubah *feature map* tiga dimensi dari lapisan *pooling* terakhir menjadi satu vektor. Vektor ini diteruskan ke *fully connected layer* (*dense*) yang berperan sebagai pengklasifikasi. Lapisan keluaran memakai aktivasi *softmax* agar menghasilkan probabilitas untuk setiap kelas:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{3}$$

dengan K jumlah kelas [10].

2.3 Regularisasi Dropout

Dropout adalah teknik regularisasi yang menonaktifkan sebagian neuron secara acak dengan probabilitas p pada setiap langkah pelatihan. Neuron yang dimatikan tidak ikut dalam

forward pass maupun *backward pass* pada langkah tersebut. Cara ini mencegah neuron saling bergantung secara berlebihan (*co-adaptation*), sehingga jaringan dipaksa mempelajari fitur yang lebih umum. Saat inferensi, semua neuron aktif kembali [11]. Di Keras, lapisan `Dropout` hanya aktif ketika model sedang dilatih [5].

2.4 Fungsi Loss, Optimizer, dan Metrik Evaluasi

Untuk klasifikasi multi-kelas, fungsi kerugian yang dipakai adalah *categorical cross-entropy*:

$$\mathcal{L} = -\sum_{i=1}^{K} y_i \log(\hat{y}_i) \tag{4}$$

dengan y_i label aktual dalam bentuk *one-hot* dan $\hat{y}_i$ probabilitas prediksi. Varian `sparse_categorical_crossentropy` menghitung hal yang sama tetapi menerima label berupa bilangan bulat. Bobot diperbarui oleh *optimizer* seperti RMSprop atau Adam [2].

Performa klasifikasi saya ukur dengan *accuracy*, *precision*, *recall*, dan *F1-score* untuk setiap kelas, ditambah *confusion matrix* yang membandingkan label aktual dengan label prediksi:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{5}$$

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{6}$$

Alat dan Bahan

3.1 Perangkat Keras

1. Laptop dengan prosesor 12 *thread* dan RAM 14 GB.
2. Koneksi internet untuk mengunduh dataset.

3.2 Perangkat Lunak

1. Sistem operasi Linux 64-bit.
2. Jupyter Notebook lokal (notebook juga dapat dijalankan di Google Colaboratory).
3. Python 3.12.
4. Pustaka Python:
    - `tensorflow` 2.21 (versi CPU) dan `keras` 3.15: membangun dan melatih CNN.
    - `numpy`: manipulasi *array* citra.
    - `matplotlib` dan `seaborn`: menampilkan citra, grafik, dan *heatmap*.
    - `scikit-learn`: `classification_report` dan `confusion_matrix`.

3.3 Dataset

1. Dataset *Dog vs Cat* dari modul, berisi 100 citra JPEG (50 kucing dan 50 anjing).
2. Dataset CIFAR-10 [6], berisi 60.000 citra berwarna 32×32 piksel dalam 10 kelas, dimuat lewat `tf.keras.datasets.cifar10`.

Hasil dan Pembahasan

Seluruh percobaan saya jalankan di Jupyter Notebook lokal memakai CPU. Notebook lengkap beserta keluarannya dapat dibuka di Google Colaboratory melalui tautan berikut:

https://colab.research.google.com/github/hafidzrizqullahprasetya/PPD/blob/main/pertemuan-9/praktikum-9-convolutional-neural-network.ipynb [1]

4.1 Percobaan Modul: Klasifikasi Citra Dog vs Cat

4.1.1 Unduh Dataset

Modul memakai perintah `gdown` di Colab. Supaya notebook bisa jalan di Colab maupun lokal tanpa paket tambahan, saya mengunduh berkas ZIP dengan `urllib`, mengekstraknya, lalu memindahkan citra ke folder `dataset/cat` dan `dataset/dog`.

```
import os
import shutil
import zipfile
import urllib.request

DATA_URL = "https://drive.usercontent.google.com/download?id=1hOU0OcYc8_za_0SJLc6HALjVc4sfVGNA&export=download&confirm=t"
urllib.request.urlretrieve(DATA_URL, "datasets.zip")
with zipfile.ZipFile("datasets.zip") as zf:
    zf.extractall(".")

for kelas in ["cat", "dog"]:
    os.makedirs(f"dataset/{kelas}", exist_ok=True)
    for nama in os.listdir(f"cnn/dataset/{kelas}"):
        shutil.move(f"cnn/dataset/{kelas}/{nama}", f"dataset/{kelas}/{nama}")
shutil.rmtree("cnn")

for kelas in sorted(os.listdir("dataset")):
    print(kelas, len(os.listdir(f"dataset/{kelas}")), "citra")
```

```
cat 50 citra
dog 50 citra
```

Gambar 1: Kode unduh dan ekstraksi dataset Dog vs Cat beserta jumlah citra per kelas.

Keluaran sel menunjukkan masing-masing kelas berisi 50 citra, jadi dataset ini seimbang.

4.1.2 Impor Pustaka

Pustaka yang saya impor sama dengan modul, yaitu `tensorflow`, `matplotlib`, `numpy`, serta lapisan `Conv2D`, `MaxPooling2D`, `Dropout`, `Flatten`, dan `Dense`. Saya menambahkan *seed* 42 untuk `random`, `numpy`, dan TensorFlow agar hasil pembagian data dan inisialisasi bobot lebih mudah diulang.

```python
import random
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense, Conv2D, MaxPooling2D, Dropout, Flatten

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
print("TensorFlow:", tf.__version__)
```

```
TensorFlow: 2.21.0
```

Gambar 2: Impor pustaka dan pengaturan *seed*.

4.1.3 Pembacaan dan Pembagian Dataset

Citra dibaca dengan `ImageDataGenerator` lalu `flow_from_directory`. Semua citra diubah ukurannya menjadi 224×224 piksel dan nilai pikselnya dibagi 255 (`rescale=1./255`) agar berada di rentang 0 sampai 1. Parameter `validation_split=0.2` membagi data menjadi 80% latih dan 20% validasi.

```python
IMAGE_SIZE = 224
BATCH_SIZE = 16

datagen = tf.keras.preprocessing.image.ImageDataGenerator(
    rescale=1. / 255,
    validation_split=0.2)

train_generator = datagen.flow_from_directory(
    base_dir, target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE, subset='training', seed=SEED)
```

Listing 1: Pembacaan dan pembagian dataset Dog vs Cat.

```python
data_path = "dataset/"
base_dir = os.path.join(data_path)

IMAGE_SIZE = 224
BATCH_SIZE = 16

datagen = tf.keras.preprocessing.image.ImageDataGenerator(
    rescale=1. / 255,
    validation_split=0.2)

train_generator = datagen.flow_from_directory(
    base_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    subset='training',
    seed=SEED)

val_generator = datagen.flow_from_directory(
    base_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    subset='validation',
    shuffle=False)

print(train_generator.class_indices)
```

```
Found 80 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
{'cat': 0, 'dog': 1}
```

Gambar 3: Pembagian dataset menjadi 80 citra latih dan 20 citra validasi.

Hasilnya 80 citra latih dan 20 citra validasi, sama dengan keluaran pada modul. Kelas

`cat` diberi indeks 0 dan `dog` indeks 1. Ukuran *batch* saya turunkan dari 64 (modul) menjadi 16 karena pelatihan berjalan di CPU dengan RAM terbatas. Satu *batch* 64 citra 224×224 dengan 32 filter menghasilkan aktivasi sekitar 400 MB per lapisan konvolusi, dan model ini punya empat lapisan konvolusi.

4.1.4 Uji Coba Pembacaan Data

Saya mengambil satu *batch* dari `train_generator` lalu menampilkan delapan citra pertama beserta labelnya.

```
img, label = next(train_generator)
print(img.shape)

nama_kelas = {v: k for k, v in train_generator.class_indices.items()}
plt.figure(figsize=(10, 5))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(img[i])
    plt.title(nama_kelas[np.argmax(label[i])])
    plt.axis('off')
plt.suptitle('Sampel citra data latih Dog vs Cat')
plt.tight_layout()
plt.show()
```

```
(16, 224, 224, 3)
```



Gambar 4: Sampel citra data latih Dog vs Cat beserta labelnya.

Bentuk *batch* yang terbaca adalah `(16, 224, 224, 3)`, artinya 16 citra berwarna 224×224. Label pada judul setiap citra sesuai dengan isi gambarnya, jadi pipeline data sudah benar. Terlihat juga bahwa latar, pencahayaan, dan pose hewan sangat beragam, sehingga tugas ini cukup sulit untuk model yang hanya dilatih dengan 80 citra.

4.1.5 Cek Bentuk Data dan IMG_SHAPE

```
for image_batch, label_batch in train_generator:
    break

print(f"Image batch shape: {image_batch.shape}")
print(f"Label batch shape: {label_batch.shape}")

IMG_SHAPE = (IMAGE_SIZE, IMAGE_SIZE, 3)
num_classes = 2
print("IMG_SHAPE:", IMG_SHAPE)
print("num_classes:", num_classes)
```

```
Image batch shape: (16, 224, 224, 3)
Label batch shape: (16, 2)
IMG_SHAPE: (224, 224, 3)
num_classes: 2
```

Gambar 5: Pemeriksaan bentuk *batch* dan pendefinisian `IMG_SHAPE`.

Bentuk label `(16, 2)` menunjukkan label sudah dalam format *one-hot* dua kelas (bawaan `class_mode='categorical'`). Karena itu lapisan keluaran memakai dua neuron *softmax* dan fungsi loss `categorical_crossentropy`. Variabel `IMG_SHAPE` saya tetapkan `(224, 224, 3)` dan `num_classes = 2`.

4.1.6 Membangun Model CNN Sederhana

Arsitektur mengikuti modul: dua blok konvolusi, masing-masing berisi dua `Conv2D` dan satu `MaxPooling2D`. Blok pertama memakai 32 filter dan diikuti `Dropout(0.1)`, blok kedua memakai 64 filter. Bagian pengklasifikasi saya ubah dari `Dense(512)` menjadi `Dense(128)` yang diikuti `Dropout(0.5)`. Alasannya, keluaran `Flatten` berukuran 200.704, sehingga `Dense(512)` akan membutuhkan sekitar 102 juta bobot dan tidak muat di memori laptop saya saat pelatihan.

```python
model = Sequential()
model.add(Input(shape=IMG_SHAPE))

model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.1))

model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))

model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
conv2d_1 (Conv2D)	(None, 224, 224, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
dropout (Dropout)	(None, 112, 112, 32)	0
conv2d_2 (Conv2D)	(None, 112, 112, 64)	18,496
conv2d_3 (Conv2D)	(None, 112, 112, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
flatten (Flatten)	(None, 200704)	0
dense (Dense)	(None, 128)	25,690,240
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258

```
Total params: 25,756,066 (98.25 MB)

Trainable params: 25,756,066 (98.25 MB)

Non-trainable params: 0 (0.00 B)
```

Gambar 6: Kode arsitektur CNN sederhana dan ringkasan `model.summary()`.

Dari ringkasan model, total parameternya 25.756.066 (98,25 MB). Yang menarik, keempat lapisan konvolusi hanya menyumbang 65.568 parameter, sedangkan lapisan `dense` pertama sendirian menyumbang 25.690.240 parameter atau sekitar 99,7% dari total. Ini membuktikan poin pada dasar teori: konvolusi dengan *weight sharing* sangat hemat parameter, dan beban terbesar justru muncul di lapisan *fully connected* setelah *flatten*.

4.1.7 Melatih Model

Model dilatih selama 20 *epoch* dengan *optimizer* RMSprop sesuai modul.

```python
epochs = 20
history = model.fit(train_generator,
                    epochs=epochs,
                    validation_data=val_generator,
                    verbose=2)
loss, acc = model.evaluate(val_generator, verbose=0)
print(f"Validation loss: {loss:.4f}")
print(f"Validation accuracy: {acc:.4f}")
```

```
Epoch 1/20
5/5 - 7s - 1s/step - accuracy: 0.4500 - loss: 1.3031 - val_accuracy: 0.5000 - val_loss: 0.6932
Epoch 2/20
5/5 - 6s - 1s/step - accuracy: 0.5125 - loss: 0.6991 - val_accuracy: 0.5000 - val_loss: 0.6906
Epoch 3/20
5/5 - 5s - 1s/step - accuracy: 0.5875 - loss: 0.6875 - val_accuracy: 0.5000 - val_loss: 0.7067
Epoch 4/20
5/5 - 6s - 1s/step - accuracy: 0.5750 - loss: 0.7066 - val_accuracy: 0.5000 - val_loss: 0.6785
Epoch 5/20
5/5 - 5s - 1s/step - accuracy: 0.5625 - loss: 0.6836 - val_accuracy: 0.5500 - val_loss: 0.6512
Epoch 6/20
5/5 - 5s - 1s/step - accuracy: 0.6625 - loss: 0.7158 - val_accuracy: 0.5000 - val_loss: 1.4529
Epoch 7/20
5/5 - 10s - 2s/step - accuracy: 0.4750 - loss: 0.8594 - val_accuracy: 0.5000 - val_loss: 0.6870
Epoch 8/20
5/5 - 5s - 1s/step - accuracy: 0.6500 - loss: 0.6539 - val_accuracy: 0.5000 - val_loss: 0.6754
Epoch 9/20
5/5 - 5s - 1s/step - accuracy: 0.6500 - loss: 0.7844 - val_accuracy: 0.5000 - val_loss: 1.0917
Epoch 10/20
5/5 - 5s - 1s/step - accuracy: 0.6875 - loss: 0.6913 - val_accuracy: 0.5500 - val_loss: 0.6394
Epoch 11/20
5/5 - 5s - 1s/step - accuracy: 0.7875 - loss: 0.5305 - val_accuracy: 0.7500 - val_loss: 0.5290
Epoch 12/20
5/5 - 5s - 1s/step - accuracy: 0.7375 - loss: 0.6745 - val_accuracy: 0.6500 - val_loss: 0.6426
Epoch 13/20
5/5 - 5s - 1s/step - accuracy: 0.8500 - loss: 0.4822 - val_accuracy: 0.8500 - val_loss: 0.4866
Epoch 14/20
5/5 - 5s - 1s/step - accuracy: 0.7750 - loss: 0.4601 - val_accuracy: 0.8000 - val_loss: 0.4997
Epoch 15/20
5/5 - 5s - 1s/step - accuracy: 0.8125 - loss: 0.3744 - val_accuracy: 0.5500 - val_loss: 0.7367
Epoch 16/20
5/5 - 5s - 1s/step - accuracy: 0.8375 - loss: 0.3050 - val_accuracy: 0.8000 - val_loss: 0.5396
Epoch 17/20
5/5 - 5s - 1s/step - accuracy: 0.9625 - loss: 0.4273 - val_accuracy: 0.5500 - val_loss: 1.8508
Epoch 18/20
5/5 - 5s - 1s/step - accuracy: 0.8000 - loss: 0.5300 - val_accuracy: 0.7500 - val_loss: 0.5471
Epoch 19/20
5/5 - 5s - 1s/step - accuracy: 0.9625 - loss: 0.1755 - val_accuracy: 0.7000 - val_loss: 0.5423
Epoch 20/20
5/5 - 5s - 1s/step - accuracy: 0.9750 - loss: 0.0999 - val_accuracy: 0.7000 - val_loss: 0.7091
Validation loss: 0.7091
Validation accuracy: 0.7000
```

Gambar 7: Log pelatihan model CNN Dog vs Cat selama 20 *epoch*.

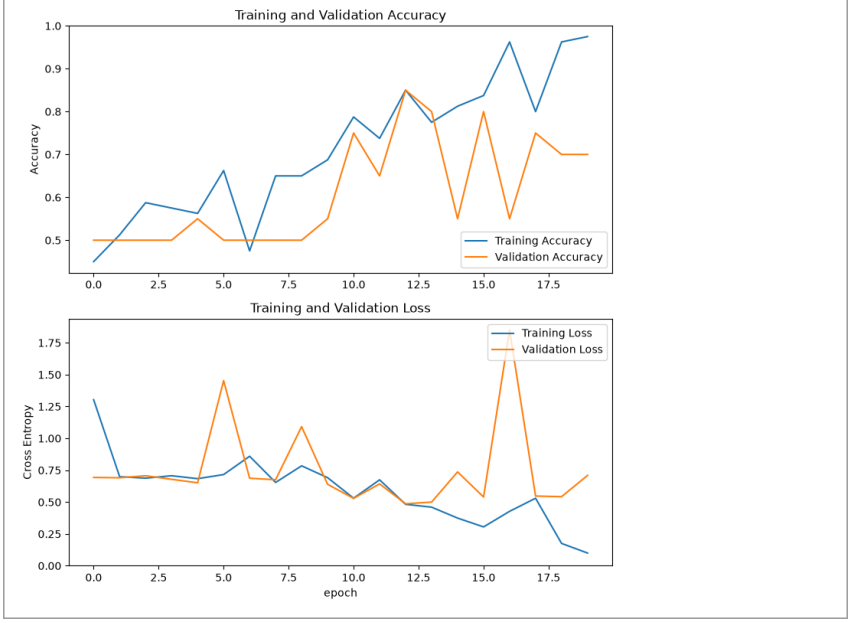
Selama 10 *epoch* pertama, akurasi validasi tertahan di 0,50 (setara tebakan acak) walaupun akurasi latih naik turun di kisaran 0,45 sampai 0,69. Model baru mulai belajar pola yang berguna pada *epoch* 11, ketika akurasi validasi naik ke 0,75, lalu mencapai nilai tertinggi 0,85 pada *epoch* 13 dengan *val_loss* 0,4866. Pada akhir pelatihan, akurasi latih mencapai 0,9750 (loss 0,0999), sedangkan akurasi validasi turun menjadi 0,70 dengan *val_loss* 0,7091.

4.1.8 Evaluasi Performa Model

```python
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure(figsize=(8, 8))
plt.subplot(2, 1, 1)
plt.plot(acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.ylabel('Accuracy')
plt.ylim([min(plt.ylim()), 1])
plt.title('Training and Validation Accuracy')

plt.subplot(2, 1, 2)
plt.plot(loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.ylabel('Cross Entropy')
plt.ylim([0, min(max(plt.ylim()), 3)])
plt.title('Training and Validation Loss')
plt.xlabel('epoch')
plt.tight_layout()
plt.show()
```



Gambar 8: Grafik akurasi dan loss data latih serta validasi model Dog vs Cat.

Grafik pada Gambar 8 memperlihatkan dua hal. Pertama, kurva validasi sangat bergerigi. Data validasi hanya 20 citra, sehingga satu citra yang salah tebak langsung menggeser akurasi sebesar 0,05; lonjakan *val_loss* ke 1,85 pada *epoch* 17 juga muncul karena alasan ini. Kedua, setelah *epoch* 13 kurva latih terus naik sedangkan kurva validasi tidak ikut membaik. Selisih akurasi 0,975 berbanding 0,70 di akhir pelatihan adalah tanda *overfitting*. Dengan 25,7 juta parameter dan hanya 80 citra latih, model mudah menghafal data latih. Menurut saya perbaikan yang paling masuk akal adalah menambah jumlah citra, memakai augmentasi data (*flip*, rotasi, *zoom*), atau mengganti `Flatten` dengan `GlobalAveragePooling2D` agar parameter lapisan *dense* turun drastis.

4.2 Tugas dan Analisis: Klasifikasi Multi-Kelas CIFAR-10

Untuk tugas, saya memilih dataset CIFAR-10 [6, 7]. Dataset ini berisi citra berwarna 32×32 piksel dari 10 kelas: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, dan *truck*. Saya memilih CIFAR-10 karena jumlah datanya cukup besar untuk melihat efek regularisasi dengan jelas, dan beberapa kelasnya mirip satu sama lain (kucing dan anjing, mobil dan truk) sehingga analisis *confusion matrix* jadi lebih menarik.

4.2.1 Memuat Dataset

```
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.layers import Rescaling
from tensorflow.keras.callbacks import EarlyStopping

(x_train_full, y_train_full), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
y_train_full = y_train_full.ravel()
y_test = y_test.ravel()

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

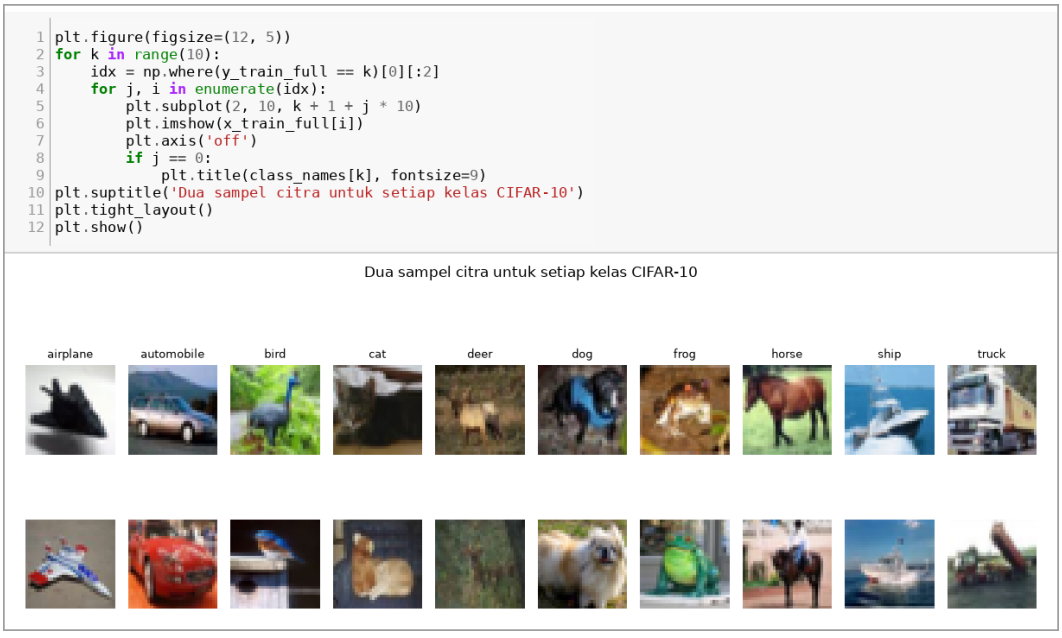
print("x_train_full:", x_train_full.shape, x_train_full.dtype)
print("x_test      :", x_test.shape)
print("Rentang piksel:", x_train_full.min(), "-", x_train_full.max())
print("Jumlah citra per kelas (latih):")
print(dict(zip(class_names, np.bincount(y_train_full))))
```

```
x_train_full: (50000, 32, 32, 3) uint8
x_test      : (10000, 32, 32, 3)
Rentang piksel: 0 - 255
Jumlah citra per kelas (latih):
{'airplane': np.int64(5000), 'automobile': np.int64(5000), 'bird': np.int64(5000), 'cat': np.int64(5000), 'deer':
 np.int64(5000), 'dog': np.int64(5000), 'frog': np.int64(5000), 'horse': np.int64(5000), 'ship': np.int64(5000),
 'truck': np.int64(5000)}
```

Gambar 9: Pemuatan dataset CIFAR-10 serta jumlah citra per kelas.

Data latih berisi 50.000 citra dan data uji 10.000 citra, bertipe `uint8` dengan rentang piksel 0 sampai 255. Setiap kelas pada data latih berisi tepat 5.000 citra, jadi dataset ini seimbang dan akurasi bisa dipakai sebagai metrik utama tanpa khawatir bias kelas mayoritas.

```
plt.figure(figsize=(12, 5))
for k in range(10):
    idx = np.where(y_train_full == k)[0][:2]
    for j, i in enumerate(idx):
        plt.subplot(2, 10, k + 1 + j * 10)
        plt.imshow(x_train_full[i])
        plt.axis('off')
        if j == 0:
            plt.title(class_names[k], fontsize=9)
plt.suptitle('Dua sampel citra untuk setiap kelas CIFAR-10')
plt.tight_layout()
plt.show()
```



Gambar 10: Dua sampel citra untuk setiap kelas CIFAR-10.

Resolusi 32×32 membuat detail objek sangat kasar. Beberapa citra bahkan sulit saya kenali sendiri tanpa melihat labelnya, terutama kelas hewan berlatar alam.

4.2.2 Pembagian Data

Dari 50.000 citra latih, saya sisihkan 5.000 citra secara acak sebagai data validasi untuk memantau pelatihan dan untuk *early stopping*. Data uji 10.000 citra hanya dipakai satu kali di tahap evaluasi akhir.

```
rng = np.random.default_rng(SEED)
perm = rng.permutation(len(x_train_full))
val_idx, train_idx = perm[:5000], perm[5000:]

x_train, y_train = x_train_full[train_idx], y_train_full[train_idx]
x_val, y_val = x_train_full[val_idx], y_train_full[val_idx]
del x_train_full, y_train_full

print("Data latih  :", x_train.shape)
print("Data validasi:", x_val.shape)
print("Data uji    :", x_test.shape)
```

```
Data latih  : (45000, 32, 32, 3)
Data validasi: (5000, 32, 32, 3)
Data uji    : (10000, 32, 32, 3)
```

Gambar 11: Pembagian data latih (45.000), validasi (5.000), dan uji (10.000).

4.2.3 Model Pembanding Tanpa Dropout

Agar pengaruh *dropout* terlihat jelas, saya membuat fungsi `bangun_cnn` yang menghasilkan arsitektur yang sama dengan dan tanpa *dropout*. Arsitekturnya terdiri atas lapisan `Rescaling(1/255)`, tiga blok konvolusi (32, 64, dan 128 filter) yang masing-masing berisi dua `Conv2D` dan satu `MaxPooling2D`, lalu `Flatten`, `Dense(256)`, dan `Dense(10)` dengan *softmax*. Rancangan ini mengikuti pola tutorial CNN TensorFlow [12]. Model dikompilasi dengan Adam dan `sparse_categorical_crossentropy` karena label CIFAR-10 berupa bilangan bulat 0 sampai 9.

```python
def bangun_cnn(dropout=True):
    m = Sequential(name='cnn_dropout' if dropout else 'cnn_tanpa_dropout')
    m.add(Input(shape=(32, 32, 3)))
    m.add(Rescaling(1. / 255))
    for f, d in [(32, 0.2), (64, 0.3), (128, 0.4)]:
        m.add(Conv2D(f, 3, padding='same', activation='relu'))
        m.add(Conv2D(f, 3, padding='same', activation='relu'))
        m.add(MaxPooling2D((2, 2)))
        if dropout:
            m.add(Dropout(d))
    m.add(Flatten())
    m.add(Dense(256, activation='relu'))
    if dropout:
        m.add(Dropout(0.5))
    m.add(Dense(10, activation='softmax'))
    m.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m
```

Listing 2: Fungsi pembangun CNN untuk CIFAR-10.

```python
def bangun_cnn(dropout=True):
    m = Sequential(name='cnn_dropout' if dropout else 'cnn_tanpa_dropout')
    m.add(Input(shape=(32, 32, 3)))
    m.add(Rescaling(1. / 255))
    for f, d in [(32, 0.2), (64, 0.3), (128, 0.4)]:
        m.add(Conv2D(f, 3, padding='same', activation='relu'))
        m.add(Conv2D(f, 3, padding='same', activation='relu'))
        m.add(MaxPooling2D((2, 2)))
        if dropout:
            m.add(Dropout(d))
    m.add(Flatten())
    m.add(Dense(256, activation='relu'))
    if dropout:
        m.add(Dropout(0.5))
    m.add(Dense(10, activation='softmax'))
    m.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m

tf.random.set_seed(SEED)
model_base = bangun_cnn(dropout=False)
hist_base = model_base.fit(x_train, y_train,
                           epochs=15,
                           batch_size=128,
                           validation_data=(x_val, y_val),
                           verbose=2)
```

```
Epoch 1/15
352/352 - 64s - 183ms/step - accuracy: 0.4177 - loss: 1.5800 - val_accuracy: 0.5564 - val_loss: 1.2270
Epoch 2/15
352/352 - 63s - 178ms/step - accuracy: 0.6233 - loss: 1.0594 - val_accuracy: 0.6594 - val_loss: 0.9569
Epoch 3/15
352/352 - 63s - 179ms/step - accuracy: 0.7020 - loss: 0.8505 - val_accuracy: 0.6830 - val_loss: 0.8951
Epoch 4/15
352/352 - 63s - 179ms/step - accuracy: 0.7521 - loss: 0.7080 - val_accuracy: 0.7240 - val_loss: 0.8048
Epoch 5/15
352/352 - 63s - 180ms/step - accuracy: 0.7878 - loss: 0.6033 - val_accuracy: 0.7372 - val_loss: 0.7870
Epoch 6/15
352/352 - 63s - 178ms/step - accuracy: 0.8171 - loss: 0.5224 - val_accuracy: 0.7270 - val_loss: 0.8165
Epoch 7/15
352/352 - 63s - 178ms/step - accuracy: 0.8392 - loss: 0.4587 - val_accuracy: 0.7324 - val_loss: 0.8608
Epoch 8/15
352/352 - 65s - 186ms/step - accuracy: 0.8562 - loss: 0.4078 - val_accuracy: 0.7398 - val_loss: 0.9077
Epoch 9/15
352/352 - 65s - 184ms/step - accuracy: 0.8749 - loss: 0.3532 - val_accuracy: 0.7376 - val_loss: 0.9634
Epoch 10/15
352/352 - 65s - 184ms/step - accuracy: 0.8945 - loss: 0.2989 - val_accuracy: 0.7310 - val_loss: 1.0628
Epoch 11/15
352/352 - 65s - 184ms/step - accuracy: 0.9086 - loss: 0.2549 - val_accuracy: 0.7316 - val_loss: 1.0909
Epoch 12/15
352/352 - 66s - 187ms/step - accuracy: 0.9190 - loss: 0.2232 - val_accuracy: 0.7432 - val_loss: 1.1698
Epoch 13/15
352/352 - 66s - 186ms/step - accuracy: 0.9339 - loss: 0.1857 - val_accuracy: 0.7262 - val_loss: 1.1971
Epoch 14/15
352/352 - 64s - 183ms/step - accuracy: 0.9467 - loss: 0.1480 - val_accuracy: 0.7288 - val_loss: 1.4277
Epoch 15/15
352/352 - 64s - 181ms/step - accuracy: 0.9559 - loss: 0.1244 - val_accuracy: 0.7304 - val_loss: 1.6207
```

Gambar 12: Pelatihan model pembanding tanpa *dropout* selama 15 *epoch*.

Model tanpa *dropout* belajar sangat cepat di data latih: akurasi latih naik dari 0,4177 pada *epoch* 1 menjadi 0,9559 pada *epoch* 15. Namun *val_loss* terendah sudah tercapai pada *epoch* 5 (0,7870). Setelah itu *val_loss* justru naik terus sampai 1,6207, sementara akurasi validasi mandek di sekitar 0,73. Pola loss latih turun tetapi loss validasi naik ini adalah gejala *overfitting* yang klasik.

4.2.4 Model CNN dengan Regularisasi Dropout

Model kedua memakai arsitektur yang sama, ditambah `Dropout` bertingkat: 0,2 setelah blok pertama, 0,3 setelah blok kedua, 0,4 setelah blok ketiga, dan 0,5 setelah `Dense(256)`. Nilai *dropout* saya buat makin besar di lapisan yang lebih dalam karena jumlah filternya lebih banyak sehingga lebih mudah menghafal. Saya juga memasang `EarlyStopping` yang memantau *val_loss* dengan `patience=5` dan `restore_best_weights=True`.

```
tf.random.set_seed(SEED)
model_cnn = bangun_cnn(dropout=True)
model_cnn.summary()
```

```
Model: "cnn_dropout"
```

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 32, 32, 3)	0
conv2d_10 (Conv2D)	(None, 32, 32, 32)	896
conv2d_11 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d_5 (MaxPooling2D)	(None, 16, 16, 32)	0
dropout_2 (Dropout)	(None, 16, 16, 32)	0
conv2d_12 (Conv2D)	(None, 16, 16, 64)	18,496
conv2d_13 (Conv2D)	(None, 16, 16, 64)	36,928
max_pooling2d_6 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_3 (Dropout)	(None, 8, 8, 64)	0
conv2d_14 (Conv2D)	(None, 8, 8, 128)	73,856
conv2d_15 (Conv2D)	(None, 8, 8, 128)	147,584
max_pooling2d_7 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_4 (Dropout)	(None, 4, 4, 128)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_4 (Dense)	(None, 256)	524,544
dropout_5 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 10)	2,570

```
Total params: 814,122 (3.11 MB)

Trainable params: 814,122 (3.11 MB)

Non-trainable params: 0 (0.00 B)
```

Gambar 13: Ringkasan arsitektur CNN dengan *dropout* untuk CIFAR-10.

Model ini hanya memiliki 814.122 parameter (3,11 MB), jauh lebih kecil dari model Dog vs Cat. Penyebabnya adalah tiga kali *max pooling* yang memperkecil *feature map* dari 32×32 menjadi $4 \times 4 \times 128$, sehingga keluaran `Flatten` hanya 2.048 nilai. Lapisan *dropout* tidak menambah parameter sama sekali.

```python
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
hist_cnn = model_cnn.fit(x_train, y_train,
                         epochs=40,
                         batch_size=128,
                         validation_data=(x_val, y_val),
                         callbacks=[early_stop],
                         verbose=2)
print("Epoch terbaik:", int(np.argmin(hist_cnn.history['val_loss'])) + 1)
```

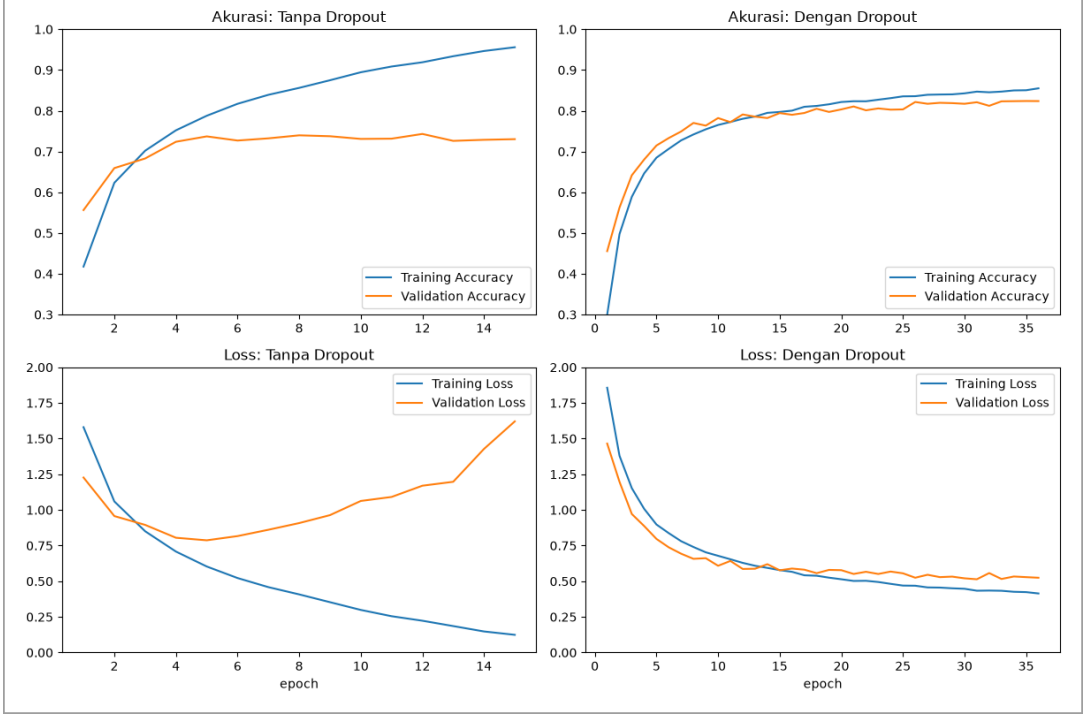
```
Epoch 1/40
352/352 - 68s - 193ms/step - accuracy: 0.2998 - loss: 1.8566 - val_accuracy: 0.4556 - val_loss: 1.4651
Epoch 2/40
352/352 - 67s - 189ms/step - accuracy: 0.4975 - loss: 1.3794 - val_accuracy: 0.5626 - val_loss: 1.1970
Epoch 3/40
352/352 - 66s - 188ms/step - accuracy: 0.5890 - loss: 1.1523 - val_accuracy: 0.6418 - val_loss: 0.9712
Epoch 4/40
352/352 - 65s - 185ms/step - accuracy: 0.6464 - loss: 1.0077 - val_accuracy: 0.6804 - val_loss: 0.8871
Epoch 5/40
352/352 - 65s - 184ms/step - accuracy: 0.6850 - loss: 0.8976 - val_accuracy: 0.7148 - val_loss: 0.7967
Epoch 6/40
352/352 - 65s - 185ms/step - accuracy: 0.7066 - loss: 0.8370 - val_accuracy: 0.7332 - val_loss: 0.7375
Epoch 7/40
352/352 - 64s - 183ms/step - accuracy: 0.7274 - loss: 0.7808 - val_accuracy: 0.7492 - val_loss: 0.6931
Epoch 8/40
352/352 - 64s - 182ms/step - accuracy: 0.7419 - loss: 0.7401 - val_accuracy: 0.7702 - val_loss: 0.6572
Epoch 9/40
352/352 - 65s - 183ms/step - accuracy: 0.7545 - loss: 0.7028 - val_accuracy: 0.7636 - val_loss: 0.6616
Epoch 10/40
352/352 - 64s - 183ms/step - accuracy: 0.7652 - loss: 0.6784 - val_accuracy: 0.7822 - val_loss: 0.6084
Epoch 11/40
352/352 - 65s - 184ms/step - accuracy: 0.7725 - loss: 0.6540 - val_accuracy: 0.7722 - val_loss: 0.6423
Epoch 12/40
352/352 - 64s - 181ms/step - accuracy: 0.7804 - loss: 0.6288 - val_accuracy: 0.7910 - val_loss: 0.5861
Epoch 13/40
352/352 - 64s - 181ms/step - accuracy: 0.7859 - loss: 0.6084 - val_accuracy: 0.7856 - val_loss: 0.5876
Epoch 14/40
352/352 - 64s - 181ms/step - accuracy: 0.7949 - loss: 0.5936 - val_accuracy: 0.7822 - val_loss: 0.6195
Epoch 15/40
352/352 - 64s - 183ms/step - accuracy: 0.7971 - loss: 0.5778 - val_accuracy: 0.7944 - val_loss: 0.5769
Epoch 16/40
352/352 - 64s - 182ms/step - accuracy: 0.8004 - loss: 0.5662 - val_accuracy: 0.7902 - val_loss: 0.5889
Epoch 17/40
352/352 - 65s - 183ms/step - accuracy: 0.8098 - loss: 0.5418 - val_accuracy: 0.7946 - val_loss: 0.5810
Epoch 18/40
352/352 - 64s - 183ms/step - accuracy: 0.8120 - loss: 0.5387 - val_accuracy: 0.8050 - val_loss: 0.5566
Epoch 19/40
352/352 - 66s - 186ms/step - accuracy: 0.8159 - loss: 0.5251 - val_accuracy: 0.7972 - val_loss: 0.5795
Epoch 20/40
352/352 - 65s - 185ms/step - accuracy: 0.8215 - loss: 0.5140 - val_accuracy: 0.8034 - val_loss: 0.5776
Epoch 21/40
352/352 - 65s - 184ms/step - accuracy: 0.8235 - loss: 0.5021 - val_accuracy: 0.8106 - val_loss: 0.5511
Epoch 22/40
352/352 - 66s - 188ms/step - accuracy: 0.8233 - loss: 0.5031 - val_accuracy: 0.8012 - val_loss: 0.5663
Epoch 23/40
352/352 - 66s - 188ms/step - accuracy: 0.8272 - loss: 0.4947 - val_accuracy: 0.8058 - val_loss: 0.5509
Epoch 24/40
352/352 - 68s - 194ms/step - accuracy: 0.8310 - loss: 0.4818 - val_accuracy: 0.8028 - val_loss: 0.5678
Epoch 25/40
352/352 - 66s - 188ms/step - accuracy: 0.8354 - loss: 0.4697 - val_accuracy: 0.8034 - val_loss: 0.5557
Epoch 26/40
352/352 - 65s - 184ms/step - accuracy: 0.8358 - loss: 0.4687 - val_accuracy: 0.8216 - val_loss: 0.5248
Epoch 27/40
352/352 - 64s - 182ms/step - accuracy: 0.8393 - loss: 0.4567 - val_accuracy: 0.8172 - val_loss: 0.5454
Epoch 28/40
352/352 - 64s - 183ms/step - accuracy: 0.8400 - loss: 0.4556 - val_accuracy: 0.8196 - val_loss: 0.5288
Epoch 29/40
352/352 - 65s - 185ms/step - accuracy: 0.8403 - loss: 0.4505 - val_accuracy: 0.8188 - val_loss: 0.5324
Epoch 30/40
352/352 - 64s - 181ms/step - accuracy: 0.8428 - loss: 0.4473 - val_accuracy: 0.8172 - val_loss: 0.5205
Epoch 31/40
352/352 - 64s - 181ms/step - accuracy: 0.8470 - loss: 0.4336 - val_accuracy: 0.8210 - val_loss: 0.5138
Epoch 32/40
352/352 - 64s - 182ms/step - accuracy: 0.8454 - loss: 0.4347 - val_accuracy: 0.8122 - val_loss: 0.5573
Epoch 33/40
352/352 - 63s - 180ms/step - accuracy: 0.8471 - loss: 0.4332 - val_accuracy: 0.8232 - val_loss: 0.5159
Epoch 34/40
352/352 - 63s - 179ms/step - accuracy: 0.8500 - loss: 0.4260 - val_accuracy: 0.8236 - val_loss: 0.5332
Epoch 35/40
352/352 - 63s - 180ms/step - accuracy: 0.8504 - loss: 0.4238 - val_accuracy: 0.8240 - val_loss: 0.5288
Epoch 36/40
352/352 - 64s - 180ms/step - accuracy: 0.8551 - loss: 0.4143 - val_accuracy: 0.8238 - val_loss: 0.5246
Epoch terbaik: 31
```

Gambar 14: Log pelatihan CNN dengan *dropout* dan *early stopping*.

Pelatihan berhenti otomatis pada *epoch* 36 karena *val_loss* tidak membaik selama lima *epoch* setelah *epoch* 31. Bobot terbaik dari *epoch* 31 (*val_loss* 0,5138, akurasi validasi 0,8210) dikembalikan ke model. Setiap *epoch* butuh sekitar 64 detik di CPU.

4.2.5 Grafik Loss dan Akurasi

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for kol, (judul, h) in enumerate([('Tanpa Dropout', hist_base), ('Dengan Dropout', hist_cnn)]):
    ep = range(1, len(h.history['loss']) + 1)
    axes[0, kol].plot(ep, h.history['accuracy'], label='Training Accuracy')
    axes[0, kol].plot(ep, h.history['val_accuracy'], label='Validation Accuracy')
    axes[0, kol].set_title(f'Akurasi: {judul}')
    axes[0, kol].set_ylim(0.3, 1.0)
    axes[0, kol].legend(loc='lower right')
    axes[1, kol].plot(ep, h.history['loss'], label='Training Loss')
    axes[1, kol].plot(ep, h.history['val_loss'], label='Validation Loss')
    axes[1, kol].set_title(f'Loss: {judul}')
    axes[1, kol].set_xlabel('epoch')
    axes[1, kol].set_ylim(0, 2.0)
    axes[1, kol].legend(loc='upper right')
plt.tight_layout()
plt.show()
```



Gambar 15: Perbandingan kurva akurasi dan loss model tanpa *dropout* (kiri) dan dengan *dropout* (kanan).

Perbedaan kedua model paling jelas terlihat di Gambar 15. Pada model tanpa *dropout* (kiri), kurva latih dan validasi berpisah sejak *epoch* 3, dan *val_loss* membentuk huruf U: turun sampai *epoch* 5 lalu naik terus. Pada model dengan *dropout* (kanan), kedua kurva berjalan berdampingan hampir sepanjang pelatihan. Akurasi validasi bahkan sempat lebih tinggi dari akurasi latih pada 10 *epoch* awal. Hal ini wajar karena akurasi latih dihitung saat *dropout* aktif (sebagian neuron mati), sedangkan validasi dihitung dengan semua neuron aktif. Konsekuensinya, model dengan *dropout* belajar lebih lambat, tetapi akurasi validasinya terus naik sampai sekitar 0,82, jauh di atas batas 0,74 yang dicapai model tanpa *dropout*.

4.2.6 Evaluasi pada Data Uji

```
hasil = {}
for nama, m in [('Tanpa Dropout', model_base), ('Dengan Dropout', model_cnn)]:
    test_loss, test_acc = m.evaluate(x_test, y_test, verbose=0)
    train_acc = m.evaluate(x_train, y_train, verbose=0)[1]
    hasil[nama] = (train_acc, test_acc, test_loss)
    print(f"{nama:15s} | akurasi latih {train_acc:.4f} | akurasi uji {test_acc:.4f} | loss uji {test_loss:.4f} | selisih {train_acc - test_acc:.4f}")
```

```
Tanpa Dropout   | akurasi latih 0.9251 | akurasi uji 0.7355 | loss uji 1.6627 | selisih 0.1896
Dengan Dropout  | akurasi latih 0.9329 | akurasi uji 0.8248 | loss uji 0.5453 | selisih 0.1081
```

```
y_prob = model_cnn.predict(x_test, verbose=0)
y_pred = np.argmax(y_prob, axis=1)
print(classification_report(y_test, y_pred, target_names=class_names, digits=4))
```

```
              precision    recall  f1-score   support

    airplane     0.8490    0.8210    0.8348      1000
  automobile     0.9370    0.9080    0.9223      1000
        bird     0.7510    0.7480    0.7495      1000
         cat     0.7176    0.6200    0.6652      1000
        deer     0.7831    0.8270    0.8045      1000
         dog     0.7598    0.7530    0.7564      1000
        frog     0.7984    0.9030    0.8475      1000
       horse     0.8535    0.8620    0.8577      1000
        ship     0.8935    0.9060    0.8997      1000
       truck     0.8982    0.9000    0.8991      1000

    accuracy                         0.8248     10000
   macro avg     0.8241    0.8248    0.8237     10000
weighted avg     0.8241    0.8248    0.8237     10000
```

Gambar 16: Perbandingan akurasi kedua model dan *classification report* model dengan *dropout* pada data uji.

Tabel 1 merangkum hasil evaluasi kedua model. Akurasi latih di tabel dihitung ulang dengan `model.evaluate` (mode inferensi, *dropout* nonaktif), sehingga bisa dibandingkan langsung dengan akurasi uji.

Tabel 1: Perbandingan performa CNN tanpa dan dengan *dropout* pada CIFAR-10.

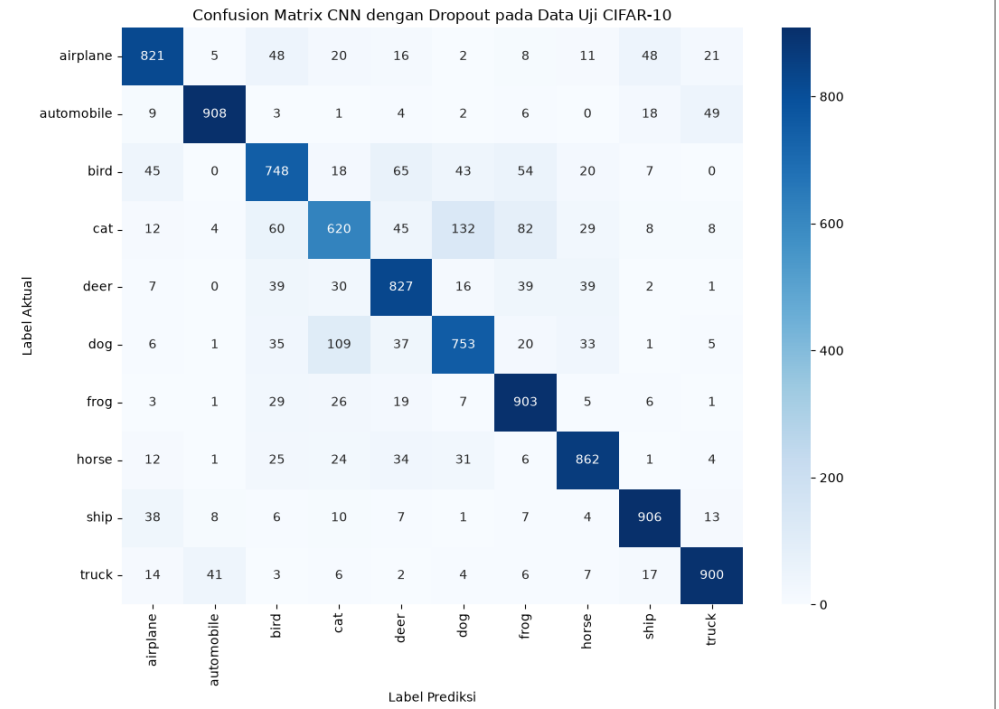
Model	**Epoch**	**Akurasi Latih**	**Akurasi Uji**	**Loss Uji**	**Selisih**
Tanpa *dropout*	15	0,9251	0,7355	1,6627	0,1896
Dengan *dropout*	36 (terbaik 31)	0,9329	**0,8248**	**0,5453**	**0,1081**

Dropout menaikkan akurasi uji dari 73,55% menjadi 82,48%, atau naik 8,93 poin persentase. Loss uji turun dari 1,6627 menjadi 0,5453. Angka loss uji model tanpa *dropout* yang tinggi menunjukkan model itu terlalu yakin pada prediksi yang salah, efek dari menghafal data latih. Selisih akurasi latih dan uji juga mengecil dari 0,1896 menjadi 0,1081.

Dari *classification report*, kelas dengan F1-*score* tertinggi adalah *automobile* (0,9223), *ship* (0,8997), dan *truck* (0,8991). Ketiganya adalah objek buatan manusia dengan bentuk dan warna yang khas. Kelas terlemah adalah *cat* dengan F1-*score* 0,6652 dan *recall* hanya 0,6200, disusul *bird* (0,7495) dan *dog* (0,7564). Nilai *macro average* dan *weighted average* sama (F1 0,8237) karena jumlah sampel tiap kelas di data uji sama, yaitu 1.000.

4.2.7 Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Label Prediksi')
plt.ylabel('Label Aktual')
plt.title('Confusion Matrix CNN dengan Dropout pada Data Uji CIFAR-10')
plt.tight_layout()
plt.show()
```



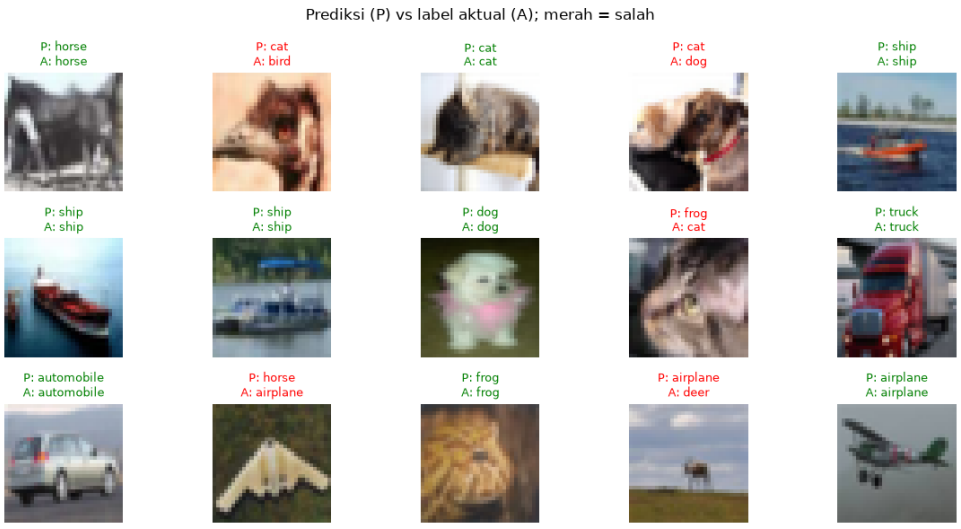
Gambar 17: *Confusion matrix* CNN dengan *dropout* pada 10.000 citra uji CIFAR-10.

Diagonal *confusion matrix* pada Gambar 17 menunjukkan jumlah prediksi benar per kelas. Kesalahan di luar diagonal paling banyak mengumpul pada pasangan kelas yang secara visual mirip:

1. ***Cat* dan *dog*:** 132 citra kucing diprediksi anjing dan 109 citra anjing diprediksi kucing. Ini pasangan kesalahan terbesar. Pada resolusi 32×32, bentuk telinga dan moncong yang membedakan keduanya hanya terwakili oleh beberapa piksel.
2. ***Automobile* dan *truck*:** 49 mobil diprediksi truk dan 41 truk diprediksi mobil. Keduanya kendaraan beroda dengan latar jalan yang serupa.
3. ***Airplane* dengan *bird* dan *ship*:** masing-masing 48 citra pesawat tertukar ke *bird* dan *ship*. Dugaan saya, model ikut memakai warna latar (langit biru atau laut) sebagai petunjuk selain bentuk objeknya.
4. **Kelas hewan berlatar alam:** *bird* cukup sering tertukar dengan *deer* (65) dan *frog* (54), dan *cat* tertukar dengan *frog* (82). Latar hijau dan cokelat yang mirip membuat fitur warna kurang bisa membedakan kelas-kelas ini.

4.2.8 Contoh Hasil Prediksi

```
idx = np.random.default_rng(7).choice(len(x_test), 15, replace=False)
plt.figure(figsize=(12, 6))
for n, i in enumerate(idx):
    plt.subplot(3, 5, n + 1)
    plt.imshow(x_test[i])
    benar = y_pred[i] == y_test[i]
    plt.title(f"P: {class_names[y_pred[i]]}\nA: {class_names[y_test[i]]}",
              color='green' if benar else 'red', fontsize=9)
    plt.axis('off')
plt.suptitle('Prediksi (P) vs label aktual (A); merah = salah')
plt.tight_layout()
plt.show()
```



Gambar 18: Contoh prediksi (P) dan label aktual (A) pada 15 citra uji; judul merah menandakan prediksi salah.

Dari 15 citra uji yang diambil acak, sebagian besar terklasifikasi benar. Prediksi yang salah umumnya terjadi pada citra yang objeknya kecil, terpotong, atau berlatar ramai, sejalan dengan pola pada *confusion matrix*.

Kesimpulan

Berdasarkan percobaan modul dan tugas yang saya kerjakan pada Pertemuan 9, kesimpulannya adalah sebagai berikut:

1. Lapisan konvolusi sangat hemat parameter karena memakai *weight sharing*. Pada model Dog vs Cat, empat lapisan `Conv2D` hanya punya 65.568 parameter, sedangkan satu lapisan `Dense` setelah `Flatten` punya 25,69 juta parameter (99,7% dari total).
2. Model CNN sederhana pada dataset Dog vs Cat mencapai akurasi validasi tertinggi 0,85 pada *epoch* 13 dan 0,70 di akhir pelatihan, sedangkan akurasi latihnya 0,975. Selisih ini menunjukkan *overfitting* akibat data latih yang hanya 80 citra dibanding parameter yang mencapai 25,7 juta.
3. Pada CIFAR-10, *dropout* bertingkat (0,2 sampai 0,5) ditambah *early stopping* menaikkan akurasi uji dari 73,55% menjadi 82,48% dan menurunkan loss uji dari 1,6627 menjadi 0,5453 dibanding arsitektur yang sama tanpa *dropout*.
4. Tanpa *dropout*, *val_loss* sudah mulai naik sejak *epoch* 5. Dengan *dropout*, kurva latih dan validasi berjalan berdampingan sampai *epoch* 31, sehingga model bisa dilatih lebih lama tanpa menghafal data.
5. *Confusion matrix* memperlihatkan bahwa kesalahan model terkumpul pada kelas yang mirip secara visual, terutama *cat* dan *dog* (241 kesalahan dua arah) serta *automobile* dan *truck* (90 kesalahan dua arah). Kelas *cat* menjadi kelas terlemah dengan F1-*score* 0,6652.

Daftar Pustaka

[1] H. R. Prasetya, "Praktikum 9: Convolutional Neural Network (Notebook)," Google Colaboratory, 2026. [Online]. Tersedia: https://colab.research.google.com/github/hafidzrizqullahprasetya/PPD/blob/main/pertemuan-9/praktikum-9-convolutional-neural-network.ipynb.

[2] I. Goodfellow, Y. Bengio, dan A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Tersedia: https://www.deeplearningbook.org/.

[3] J. Han, J. Pei, dan H. Tong, *Data Mining: Concepts and Techniques*, edisi ke-4. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[4] Keras Team, "Conv2D layer," Keras API Documentation. [Online]. Tersedia: https://keras.io/api/layers/convolution_layers/convolution2d/.

[5] Keras Team, "Dropout layer," Keras API Documentation. [Online]. Tersedia: https://keras.io/api/layers/regularization_layers/dropout/.

[6] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Tersedia: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[7] A. Krizhevsky, "The CIFAR-10 dataset," University of Toronto. [Online]. Tersedia: https://www.cs.toronto.edu/~kriz/cifar.html.

[8] A. Krizhevsky, I. Sutskever, dan G. E. Hinton, "ImageNet classification with deep convolutional neural networks," dalam *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, 2012, hlm. 1097 sampai 1105. [Online]. Tersedia: https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.

[9] Y. LeCun, L. Bottou, Y. Bengio, dan P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, hlm. 2278 sampai 2324, 1998.

[10] K. O'Shea dan R. Nash, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, 2015. [Online]. Tersedia: https://arxiv.org/abs/1511.08458.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, dan R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, hlm. 1929 sampai 1958, 2014. [Online]. Tersedia: https://jmlr.org/papers/v15/srivastava14a.html.

[12] TensorFlow, "Convolutional Neural Network (CNN)," TensorFlow Tutorials. [Online]. Tersedia: https://www.tensorflow.org/tutorials/images/cnn.